

MANUAL DE INSTALACIÓN Y DESPLIEGUE

SISTEMA DE GESTIÓN EMPRESARIAL

CONSORCIOS VILLEGAS E.I.R.L.

Guía de preparación, publicación, respaldo y recuperación



Logotipo institucional de Consorcios Villegas E.I.R.L.

Código del documento: VT-OPS-001

Repositorio: <https://github.com/JhonMM27/Villegas2026>

Rama revisada: main

Commit de referencia: 7b99815c8778

Fecha de revisión: 29/08/2026

Versión del documento: 1.1

Control del documento

Campo	Valor
Documento	Manual de Instalación y Despliegue
Código	VT-OPS-001
Sistema	Villegas2026 - Consorcios Villegas E.I.R.L.
Repositorio	https://github.com/JhonMM27/Villegas2026
Rama	main
Commit de referencia	7b99815c87786c3b5e13be5b2890a77d4b2aa9c8
Fecha de revisión	29/08/2026
Versión	1.1
Responsable	Consorcios Villegas E.I.R.L. - Administración / TI
Aprobado por	Propietario del sistema
Público objetivo	Administradores de sistemas, desarrolladores responsables de releases, soporte técnico y operaciones.

Historial de versiones

Versión	1.1	Responsable	Descripción
1.0	29/08/2026	Consorcios Villegas E.I.R.L.	Primera edición separada y completa, basada en revisión del repositorio Villegas2026.

Criterio de evidencia

Para no confundir funcionalidades implementadas con propuestas, el documento distingue tres niveles de evidencia:

Etiqueta	Significado
VERIFICADO	Existe evidencia directa en código, rutas, modelos, servicios, pruebas o documentos del repositorio.
INFERIDO	Conclusión obtenida de la relación entre varios elementos del código; debe confirmarse en el ambiente real cuando corresponda.
RECOMENDADO	Buena práctica de operación, seguridad o despliegue. No implica que esté configurada actualmente.
Seguridad documental: no incluir contraseñas, APP_KEY, tokens, certificados privados, credenciales de base de datos ni valores completos del archivo .env en capturas o anexos.	

Fuentes técnicas revisadas

ID	Fuente	Uso
R1	README.md	Identidad general del proyecto.

ID	Fuente	Uso
R2	AGENTS.md	Stack, convenciones, servicios, reglas de kardex e inmutabilidad.
R3	routes/web.php	Mapa funcional, rutas y módulos.
R4	composer.json	Dependencias PHP y scripts Composer.
R5	package.json	Dependencias frontend y scripts Vite.
R6	app/Models	Entidades y relaciones Eloquent.
R7	app/Services	Lógica de negocio y servicios de dominio.
R8	database/migrations	Estructura y evolución del esquema.
R9	database/seeder/RolesAndPermissionsSeeder.php	Roles, permisos y provisión inicial.
R10	tests/Feature y tests/Unit	Cobertura de pruebas existente; no se presume ejecución exitosa.
R11	PLANILLA.md	Funcionamiento documentado del módulo de planilla.

Referencias externas usadas únicamente para recomendaciones de documentación/despliegue:

Referencia	Aplicación
ISO/IEC/IEEE 15289:2019	Estructura de información del ciclo de vida de sistemas y software.
ISO/IEC/IEEE 26514:2022	Diseño y desarrollo de información para usuarios de software.
Laravel 12 Documentation	Buenas prácticas de instalación, configuración y despliegue.
C4 Model	Convención recomendada para diagramas de arquitectura.

Índice del documento

Índice materializado sin número de página porque la paginación cambiará al reemplazar los recuadros por capturas. Al finalizar, en Microsoft Word inserte o actualice una Tabla de contenido automática.

Sección	Contenido
1	Objetivo y alcance
2	Resumen del entorno tecnológico
3	Arquitectura de ejecución
4	Prerrequisitos
5	Obtención del código
6	Instalación local / desarrollo
7	Configuración segura de .env
8	Base de datos y seeders

Sección	Contenido
9	Ejecución de desarrollo
10	Compilación frontend
11	Pruebas y calidad
12	Preparación de producción
13	Servidor web y PHP
14	Permisos y almacenamiento
15	Despliegue de una versión
16	Verificación posterior al despliegue
17	Backups
18	Restauración y recuperación
19	Mantenimiento del kardex
20	Actualización y rollback
21	Logs y diagnóstico
22	Seguridad operacional
23	Solución de problemas
24	Rutina de mantenimiento
Anexo A	Comandos de referencia
Anexo B	Checklist de puesta en producción
Anexo C	Evidencias a insertar

1. Objetivo y alcance

Este manual describe el procedimiento para instalar, configurar, ejecutar, compilar y desplegar Villegas2026, además de establecer una guía operativa para respaldos, recuperación, actualización y validación posterior a un release. Los pasos de desarrollo se basan en los scripts y convenciones existentes en el repositorio. Las configuraciones de infraestructura productiva se marcan como RECOMENDADAS cuando el repositorio no define el servidor real.

No copie valores de producción desde este documento. Use variables de entorno y un gestor seguro de secretos. El repositorio revisado contiene un archivo .env versionado; antes de un despliegue real debe retirarse del control de versiones y rotarse cualquier secreto que haya quedado expuesto.

2. Resumen del entorno tecnológico

Componente	Versión / evidencia	Estado
Backend	Laravel 12.0 sobre PHP ^8.2	VERIFICADO en composer.json / AGENTS.md
Base de datos	MySQL	VERIFICADO
Frontend	Blade + Tailwind CSS 4 + Vite	VERIFICADO
Build frontend	Vite 6.2.x	VERIFICADO en package.json
Permisos	spatie/laravel-permission ^6.24	VERIFICADO
PDF	barryvdh/laravel-dompdf ^3.1	VERIFICADO
Excel	maatwebsite/excel ^3.1	VERIFICADO
Tablas AJAX	yajra/laravel-datatables-oracle ^12	VERIFICADO
Pruebas	PHPUnit 11.5.x	VERIFICADO como dependencia dev

2.1. Scripts disponibles

Comando	Uso
composer run dev	Arranca servidor Laravel, queue:listen, Pail y Vite en desarrollo.
composer test	Limpia la configuración y ejecuta la suite de pruebas.
npm run dev	Servidor de desarrollo Vite.
npm run build	Compilación de assets para publicación.

3. Arquitectura de ejecución

En desarrollo, la aplicación se ejecuta como un monolito Laravel: el navegador consume rutas web, los controladores orquestan servicios de negocio, Eloquent persiste en MySQL y Blade/Vite entregan la interfaz. En producción, el repositorio no documenta una topología definitiva de servidor web; por ello la figura siguiente debe adaptarse al ambiente real.

CAPTURA PENDIENTE OPS-01
Diagrama de despliegue real
Reemplazar este recuadro con una captura anonimizada del ambiente autorizado.

4. Prerrequisitos

Requisito	Criterio
Git	Necesario para clonar/actualizar el repositorio.
PHP	8.2 o superior según composer.json.
Composer	Compatible con las dependencias del proyecto.
MySQL	Servidor accesible y una base de datos para Villegas2026.
Node.js + npm	Versión compatible con Vite 6 y las dependencias de package.json.
Extensiones PHP	Ejecutar composer install para validar las extensiones exigidas por las dependencias en el servidor objetivo.
Espacio y permisos	El usuario del proceso PHP debe poder escribir en storage y bootstrap/cache.
No se declara en el repositorio una versión exacta de MySQL, Node.js, Nginx o Apache. Defina y registre las versiones que se usen realmente en producción antes de aprobar este manual.	

5. Obtención del código

1. Abrir una terminal en el directorio de trabajo.
2. Clonar el repositorio.
3. Entrar al directorio Villegas2026.
4. Confirmar que se está trabajando en la rama y commit autorizados para el despliegue.

```
git clone https://github.com/JhonMM27/Villegas2026.git
cd Villegas2026
git checkout main
git rev-parse HEAD
```

La revisión documental de esta edición usa como referencia el commit 7b99815c87786c3b5e13be5b2890a77d4b2aa9c8. Para un release posterior, sustituya el SHA por el commit aprobado y actualice la portada.

6. Instalación local / desarrollo

6.1. Preparación inicial recomendada

El proyecto no define un script Composer de instalación integral. La preparación inicial debe realizarse mediante el procedimiento manual y controlado descrito a continuación.

```
composer install
# Crear y configurar .env de forma segura
php artisan key:generate
php artisan migrate
npm install
npm run build
```

Antes de ejecutar migraciones, confirme si la base es nueva o existente y obtenga un respaldo cuando corresponda.

6.2. Opción manual y controlada

1. Instalar dependencias PHP con Composer.
2. Crear manualmente un archivo .env seguro para el ambiente; el repositorio no incluye .env.example.
3. Generar APP_KEY.
4. Configurar la conexión MySQL sin documentar secretos.
5. Ejecutar migraciones y, cuando corresponda, seeders.
6. Instalar dependencias frontend.
7. Compilar o iniciar Vite.

```
composer install
# Crear .env de forma segura según el ambiente
php artisan key:generate
php artisan migrate
php artisan db:seed
npm install
npm run build
```

7. Configuración segura de .env

El archivo .env debe existir únicamente en cada ambiente de ejecución y no debe convertirse en documentación de credenciales. Para la conexión MySQL se documentan los nombres de variables, no sus valores reales:

```
APP_ENV=local|production
APP_DEBUG=true|false
APP_URL=https://dominio-ejemplo

DB_CONNECTION=mysql
DB_HOST=...
DB_PORT=3306
DB_DATABASE=...
DB_USERNAME=...
DB_PASSWORD=...
```

Variable / grupo	Criterio operativo
------------------	--------------------

Variable / grupo	Criterio operativo
APP_KEY	Generar con php artisan key:generate. No copiar una clave entre proyectos sin una política explícita.
APP_ENV	local en desarrollo; production en producción.
APP_DEBUG	false en producción.
APP_URL	URL canónica del ambiente.
DB_*	Cuenta de base de datos con privilegios mínimos necesarios. No usar credenciales personales.
Mail/servicios externos	Configurar solo si el ambiente realmente los utiliza; almacenar secretos fuera del repositorio.
Hallazgo VERIFICADO: el árbol del repositorio de referencia contiene un archivo .env. Acción requerida antes de producción: retirarlo del historial/control de versiones según el procedimiento de seguridad de la organización y rotar las credenciales/secretos que hayan podido exponerse. Este manual no reproduce su contenido.	

8. Base de datos y seeders

8.1. Preparación de MySQL

1. Crear una base de datos vacía para el ambiente.
2. Crear un usuario técnico con permisos sobre esa base.
3. Configurar DB_* en .env.
4. Probar conexión con php artisan migrate:status o ejecutar las migraciones en un ambiente nuevo.

CAPTURA PENDIENTE OPS-03 Base de datos preparada Reemplazar este recuadro con una captura anonimizada del ambiente autorizado.
--

8.2. Migraciones y datos iniciales

```
php artisan migrate
php artisan db:seed
```

El repositorio incluye seeders para unidades, afectaciones, tipos de operación, formas/medios de pago, datos nutricionales y roles/permisos, entre otros. RolesAndPermissionsSeeder crea roles admin, vendedor y operador y genera permisos CRUD por dominio, además de permisos de reportes.

El seeder de roles también provisiona una cuenta administrativa de desarrollo con credenciales definidas en código. No se reproducen esas credenciales aquí. Antes de producción, elimine esa provisión, cambie las credenciales y valide que no permanezca ninguna contraseña predeterminada.

8.3. Instalación desde cero vs. actualización

Escenario	Comando / criterio
Base nueva	php artisan migrate && php artisan db:seed, previa validación del responsable.
Base existente	php artisan migrate; NO usar migrate:fresh porque elimina tablas y datos.
Pruebas aisladas	migrate:fresh --seed puede usarse únicamente en una base desechable de pruebas.
Producción	Respaldar antes de migrar y usar php artisan migrate --force dentro del runbook aprobado.

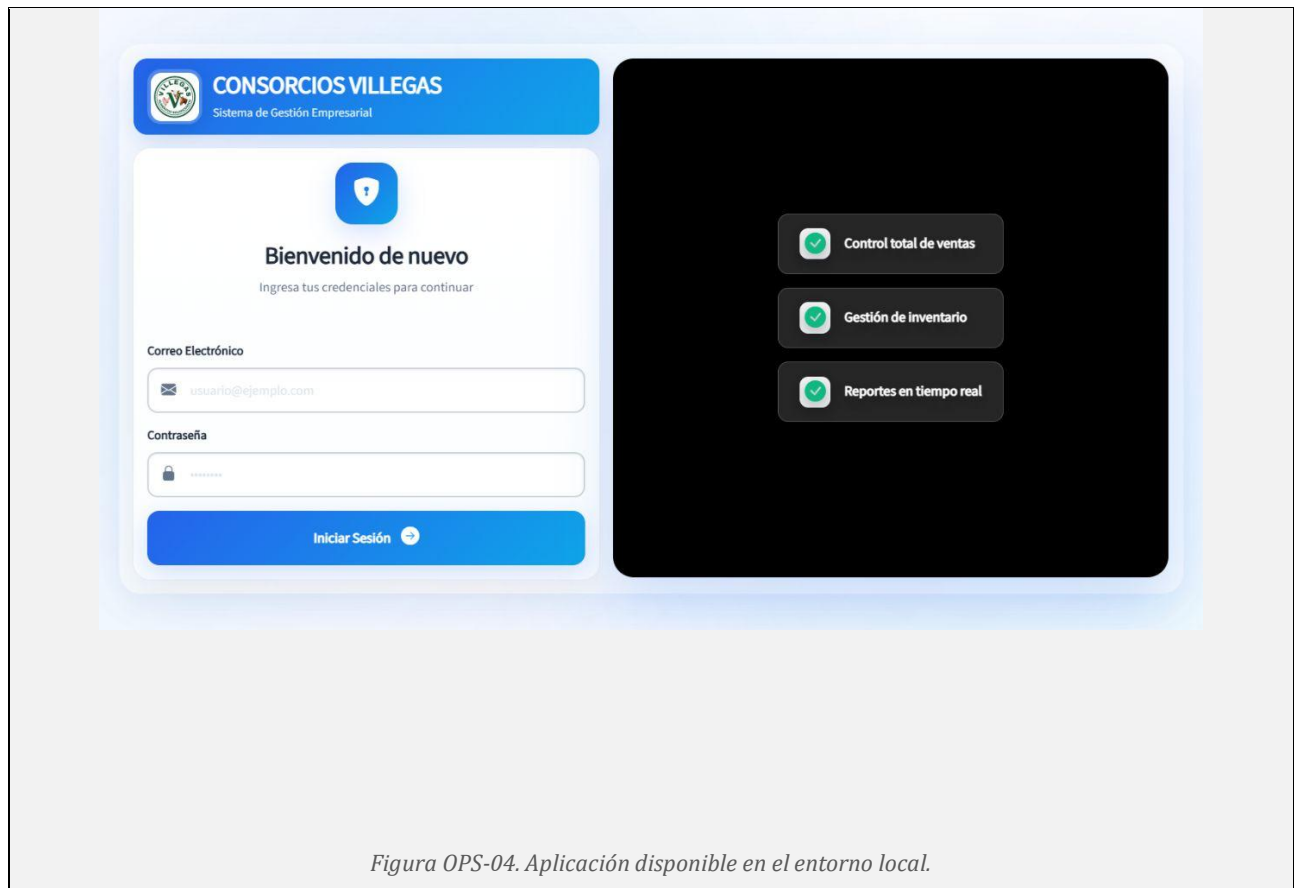
9. Ejecución de desarrollo

El repositorio documenta tres formas de trabajo. La opción integral es composer run dev, que inicia procesos concurrentes.

```
# Opción integral
composer run dev
```

```
# Opción separada
php artisan serve
npm run dev
```

composer run dev ejecuta el servidor de Laravel, queue:listen --tries=1 y Vite. La presencia del listener en el script no demuestra que todos los módulos dependan de cola; confirme trabajos encolados antes de diseñar un supervisor productivo.



10. Compilación frontend

```
npm install
# o en CI/release reproducible, si package-lock está actualizado:
npm ci

npm run build
```

El build genera los assets que Vite entrega en producción. Después de compilar, confirme que el manifest y los archivos generados estén disponibles para el servidor de Laravel.

11. Pruebas y calidad

```
php artisan test
# equivalente definido por Composer
composer test

./vendor/bin/pint --test
```

El repositorio contiene pruebas Feature y Unit sobre auditoría, cuentas corrientes, sueldo de empleados, cronología del MovimientoService, núcleos, reportes de planilla, rectificación de ventas, pago inicial y auditoría de rectificaciones. Esta documentación registra su existencia, pero no afirma que estén aprobadas en el ambiente de entrega porque no se ejecutaron contra la base de datos del usuario.

Evidencia mínima antes de release	Resultado a registrar
php artisan test	Total, aprobadas, fallidas y fecha.

Evidencia mínima antes de release	Resultado a registrar
./vendor/bin/pint --test	Sin diferencias de estilo o excepciones justificadas.
npm run build	Compilación finalizada sin error.
php artisan migrate:status	Migraciones esperadas aplicadas.
php artisan kardex:validar	Sin ruptura de cadena, cuando el release afecta inventario/kardex.

12. Preparación de producción

La siguiente lista es RECOMENDADA porque el repositorio no contiene un manifiesto de infraestructura productiva completo:

- Servidor Linux o plataforma equivalente con PHP 8.2+ y extensiones exigidas por Composer.
- Servidor MySQL accesible únicamente desde los orígenes autorizados.
- Servidor web configurado para usar la carpeta public como document root.
- TLS/HTTPS y renovación de certificados.
- APP_ENV=production y APP_DEBUG=false.
- Credenciales y secretos fuera de Git.
- Cuenta técnica del proceso PHP con permisos mínimos.
- Backups automáticos verificados mediante restauraciones de prueba.
- Monitoreo de espacio, errores HTTP, logs de Laravel y disponibilidad de MySQL.
- Procedimiento de rollback documentado antes de cada cambio de esquema.

13. Servidor web y PHP

Laravel debe publicarse desde la carpeta public. El siguiente bloque es un EJEMPLO de Nginx, no evidencia de que Nginx sea el servidor utilizado actualmente. Ajuste dominio, socket PHP-FPM, límites y cabeceras al ambiente real.

```
server {
    listen 443 ssl http2;
    server_name sistema.ejemplo.local;
    root /var/www/Villegas2026/public;
    index index.php;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_pass unix:/run/php/php8.2-fpm.sock;
    }
}
```

No copie este bloque sin validarlo. Si el ambiente real usa Apache, contenedores, IIS u otro servicio administrado, documente esa topología en OPS-01 y sustituya este ejemplo.

14. Permisos y almacenamiento

Laravel necesita escritura en storage y bootstrap/cache. La identidad exacta del usuario/grupo depende del servidor. Evite chmod 777. Ejemplo conceptual:

```
sudo chown -R <usuario_php>:<grupo_php> storage bootstrap/cache
sudo chmod -R u+rwX,g+rwX storage bootstrap/cache
```

Si el sistema guarda archivos públicos mediante el filesystem de Laravel, verifique también el enlace public/storage según la configuración real. No se debe asumir su necesidad si el módulo no lo utiliza.

15. Despliegue de una versión

Runbook recomendado para una actualización controlada de producción:

1. Registrar ticket/release, commit objetivo y responsable.
2. Comunicar ventana de mantenimiento y detener temporalmente operaciones sensibles cuando el cambio afecte inventario, compras, ventas, preparadas, núcleos o préstamos.
3. Obtener respaldo consistente de MySQL y, si aplica, de archivos cargados.
4. Validar el kardex antes del cambio si el release toca inventario.
5. Actualizar el código al commit aprobado.
6. Instalar dependencias PHP sin paquetes de desarrollo.
7. Instalar/compilar assets frontend.
8. Ejecutar migraciones pendientes con respaldo confirmado.
9. Limpiar/reconstruir cachés necesarias.
10. Levantar la aplicación y ejecutar smoke tests.
11. Validar logs y dejar evidencia de versión/commit desplegado.

```
# Ejemplo de secuencia; adaptar a su CI/CD
php artisan down

git fetch --all --prune
git checkout <commit-aprobado>
composer install --no-dev --optimize-autoloader
npm ci
npm run build
php artisan migrate --force
php artisan optimize:clear

php artisan up
```

php artisan down/up se presenta como mecanismo recomendado para una ventana controlada. Si la infraestructura usa balanceadores, blue/green o despliegues sin interrupción, reemplace esta secuencia por el procedimiento real.

16. Verificación posterior al despliegue

Prueba	Resultado esperado	Evidencia
Acceso HTTPS / login	Pantalla carga sin error y autenticación funciona para una cuenta autorizada.	OPS-05

Prueba	Resultado esperado	Evidencia
Dashboard	Carga totales/alertas sin excepción.	OPS-05
Permisos	Usuario no administrador solo ve/ejecuta lo permitido.	Registro de prueba
Productos / clientes / proveedores	Listados y búsquedas cargan.	Checklist
Ventas/compras	Consultar documentos existentes. Evitar crear una transacción real solo para probar.	Checklist
Reportes PDF/Excel	Generación de una muestra controlada.	Archivo de evidencia
Logs	Sin errores 500 nuevos asociados al release.	Registro de verificación
Kardex	php artisan kardex:validar cuando aplique.	OPS-07
CAPTURA PENDIENTE OPS-05 Verificación funcional posterior al despliegue Reemplazar este recuadro con una captura anonimizada del ambiente autorizado.		

17. Backups

17.1. Base de datos

La política concreta de backup no está definida en el repositorio. Como mínimo, el responsable debe establecer frecuencia, retención, cifrado, almacenamiento externo y pruebas de restauración. Ejemplo genérico de dump consistente:

```
mysqldump --single-transaction --routines --triggers \
-u <usuario_backup> -p <base_datos> \
> villegas_$(date +%Y%m%d_%H%M%S).sql
```

17.2. Archivos

Respalidar únicamente los directorios con información no regenerable (por ejemplo, uploads reales), además de la configuración segura gestionada fuera de Git. vendor y node_modules no requieren backup porque se reconstruyen desde los manifiestos de dependencias.

CAPTURA PENDIENTE OPS-06
Evidencia de backup exitoso
Reemplazar este recuadro con una captura anonimizada del ambiente autorizado.

18. Restauración y recuperación

1. Declarar incidente y detener escrituras en la aplicación.
2. Seleccionar un backup validado acorde al punto de recuperación requerido.
3. Restaurar primero en un ambiente aislado cuando sea posible.
4. Verificar migraciones, integridad de datos y acceso a la aplicación.
5. Ejecutar controles de kardex si el incidente afecta inventario.
6. Validar reportes críticos y saldos.
7. Documentar fecha de restauración, backup utilizado, responsable y resultado.
8. Reabrir el sistema únicamente tras aprobación.

```
mysql -u <usuario_restore> -p <base_datos> < backup_validado.sql
```

18.1. RPO y RTO

Métrica	Definición	Valor aprobado
RPO	Máxima pérdida de datos aceptable medida en tiempo.	Por aprobar en la política operativa del propietario
RTO	Tiempo máximo objetivo para recuperar el servicio.	Por aprobar en la política operativa del propietario

19. Mantenimiento del kardex

El inventario es un componente crítico. AGENTS.md establece que los cambios de stock deben pasar por MovimientoService y que no se debe corregir stock_almacen manualmente. Los comandos de diagnóstico registrados en el proyecto son:

```
php artisan kardex:validar
php artisan kardex:validar --producto_id=<ID>
php artisan kardex:recalcular <producto_id> <YYYY-MM-DD>
```

- registrarIngreso() y registrarSalida() detectan movimientos retroactivos y disparan recálculo cronológico cuando corresponde.
- El kardex conserva stock/costo/valor anterior y nuevo para mantener trazabilidad del CPP.
- Las rectificaciones con fechas pasadas deben respetar el orden fecha + id.

CAPTURA PENDIENTE OPS-07

Validación del kardex

Reemplazar este recuadro con una captura anonimizada del ambiente autorizado.

20. Actualización y rollback

20.1. Actualización

- Usar un commit/tag identificado y aprobado.
- Revisar migraciones incluidas antes de ejecutar.
- Respalidar la base de datos.
- Compilar assets con las dependencias bloqueadas.
- Ejecutar pruebas y smoke tests.
- Registrar quién desplegó, cuándo y qué commit quedó activo.

20.2. Rollback

Un rollback de código no siempre revierte automáticamente un cambio de esquema o de datos. La decisión depende de las migraciones del release. El plan mínimo debe contemplar:

1. Detener escrituras.
2. Determinar si basta volver al commit anterior o si es necesario restaurar base de datos.
3. Restaurar backup cuando exista incompatibilidad de datos/esquema.
4. Recompilar assets de la versión anterior.
5. Validar login, dashboard, permisos, reportes e inventario.
6. Documentar causa raíz y evidencias.

21. Logs y diagnóstico

Laravel Pail está disponible como dependencia de desarrollo, pero no forma parte del script composer run dev actual. En un servidor productivo, defina una política de rotación y acceso para logs. Los logs no deben exponerse públicamente.

Síntoma	Revisión inicial
HTTP 500	storage/logs, permisos de storage/bootstrap/cache, APP_DEBUG=false y correlación con hora del incidente.
Error de conexión MySQL	DB_HOST/PORT/DB_DATABASE/DB_USERNAME, red y privilegios.
Assets sin estilos/JS	Verificar npm run build y existencia del manifest generado por Vite.

Síntoma	Revisión inicial
403 / acción no autorizada	Rol y permisos Spatie; limpiar caché de permisos si el flujo de administración lo requiere.
Datos viejos tras despliegue	php artisan optimize:clear y revisar caché de aplicación.
PDF/Excel falla	Revisar dependencias, permisos, memoria y logs; DomPDF/Maatwebsite Excel están instalados por Composer.

22. Seguridad operacional

Control	Acción requerida
Secretos	Eliminar .env de Git; rotar secretos potencialmente expuestos; usar almacenamiento seguro.
Cuenta predeterminada	Cambiar/eliminar cuenta administrativa de desarrollo creada por seeders.
HTTPS	Forzar TLS en producción.
Debug	APP_DEBUG=false.
Permisos	Aplicar mínimo privilegio a roles del sistema, usuario MySQL y usuario del proceso PHP.
Backups	Cifrar/proteger, limitar acceso y probar restauración.
Logs	No registrar ni publicar contraseñas/tokens; limitar acceso.
Dependencias	Actualizar con revisión y ejecutar pruebas antes de release.
Repositorio	Proteger la rama de producción y usar revisión de cambios. La rama main observada no aparece protegida en la consulta de GitHub realizada para esta edición.

23. Solución de problemas

Problema	Causa probable	Acción
No existe APP_KEY	Instalación incompleta	php artisan key:generate en el ambiente correcto.
SQLSTATE / connection refused	Conexión DB incorrecta o MySQL no disponible	Revisar DB_* y conectividad; no copiar credenciales al ticket.
Migraciones pendientes	Release no completado	php artisan migrate:status; aplicar migración aprobada con backup.
Vite manifest not found	Frontend no compilado	npm install/npm ci y npm run build.
Permission denied	Permisos de filesystem	Corregir propietario/permisos de storage y bootstrap/cache sin usar 777.

Problema	Causa probable	Acción
Usuario no ve módulo	Rol/permisos	Validar asignación Spatie y permisos del dominio.
Reporte no descarga	Error PDF/Excel o datos/filtros	Revisar logs, filtros y dependencias; repetir en preproducción.
Cambio no refleja	Caché/asset	php artisan optimize:clear y recompilar si corresponde.

24. Rutina de mantenimiento

Frecuencia sugerida	Actividad	Evidencia
Diaria	Revisar disponibilidad, errores críticos, espacio y estado de backup.	Registro operativo.
Semanal	Revisar fallos repetidos, capacidad y permisos/cuentas pendientes.	Checklist semanal.
Mensual	Probar restauración de una copia en ambiente aislado; revisar dependencias y tamaño de BD/logs.	Acta de prueba.
Antes de release	Backup, tests, build, migraciones revisadas, smoke test y plan de rollback.	Checklist de release.

Anexo A. Comandos de referencia

Área	Comando
Instalación	composer install
Desarrollo	composer run dev
Servidor Laravel	php artisan serve
Vite	npm run dev
Build	npm run build
Migraciones	php artisan migrate
Seeders	php artisan db:seed
Pruebas	php artisan test / composer test
Estilo	./vendor/bin/pint --test
Limpiar cachés	php artisan optimize:clear
Kardex validar	php artisan kardex:validar
Kardex recalcular	php artisan kardex:recalcular {producto_id} {fecha}

Anexo B. Checklist de puesta en producción

Verificación	Estado
Código/commit de release identificado	☐
Backup de MySQL generado y verificado	☐
Rollback definido	☐
.env fuera de control de versiones	☐
Secretos rotados/seguros	☐
Cuenta administrativa predeterminada cambiada/eliminada	☐
APP_ENV=production	☐
APP_DEBUG=false	☐
HTTPS activo	☐
composer install --no-dev completado	☐
npm run build completado	☐
Migraciones revisadas/aplicadas	☐
Pruebas ejecutadas y registradas	☐
Permisos filesystem correctos	☐
Smoke test funcional completado	☐
kardex:validar completado cuando aplica	☐
Logs revisados	☐
Responsable y fecha del despliegue registrados	☐

Anexo C. Evidencias e imágenes a insertar

ID	Evidencia	Qué debe mostrar
OPS-01	Diagrama de despliegue real	Topología real del ambiente.
OPS-02	Instalación de dependencias	Terminal al finalizar instalación.
OPS-03	Base de datos preparada	MySQL/migraciones sin credenciales.
OPS-04	Desarrollo ejecutándose	Laravel/Vite y login local.
OPS-05	Smoke test posterior	Dashboard funcional después del release.
OPS-06	Backup exitoso	Archivo, fecha, tamaño y estado.
OPS-07	Validación de kardex	Salida de kardex:validar.

Regla: todas las capturas deben anonimizar credenciales y datos personales. Conserve el ID OPS-xx como pie de figura.

APROBACIÓN Y CONTROL DE ENTREGA

El presente manual de instalación y despliegue - villegas2026 corresponde a la versión 1.1, revisada el 29/08/2026. Su aprobación se formaliza mediante las firmas de los responsables indicados a continuación.

Rol	Responsable	Firma y fecha
Elaborado / actualizado por	Administración / TI	
Revisado por	Responsable funcional	
Aprobado por	Propietario del sistema	

Nota de control: las capturas identificadas como pendientes pueden incorporarse posteriormente sin alterar el contenido funcional aprobado, siempre que correspondan a la misma versión del sistema y no expongan datos personales ni credenciales.